École nationale supérieure d'intelligence artificielle et science de données
(ensiasd)

Projet Académique

Taroudant Heritage Monitoring

Monitoring et valorisation du patrimoine de Taroudant

Réalisé par :
Aimad Bouya
Hiba Bouzid
Khadija Bourais

Encadrante :
Sara El-Atief

18 avril 2026

Résumé

Le suivi du patrimoine bâti de Taroudant repose actuellement sur des procédures manuelles, longues et sujettes à erreurs. Ce projet propose une solution numérique centralisant les données de chaque monument, automatisant la collecte d'informations et fournissant des visualisations en temps réel. L'application permet ainsi de réduire considérablement le temps d'inspection, d'améliorer la traçabilité des interventions et d'offrir aux décideurs une vision claire et actualisée du patrimoine.

Table des matières

1 Introduction **3**

2 Présentation du projet **3**

3 Architecture du système **3**

3.1 Frontend 3

3.2 Backend 4

3.3 Base de données 4

4 Modèle Conceptuel de Données (DCM) **4**

5 Points de terminaison de l'API **4**

5.1 Authentification 4

5.2 Monuments 5

5.3 Inspections 5

5.4 Fissures (Cracks) 5

5.5 Rapports 5

5.6 Utilisateurs et Administration 5

5.7 Notifications et Affectations 5

5.8 Demandes d'accès 6

5.9 Analytiques 6

6 Sécurité **6**

7 Diagrammes Techniques **7**

7.1 Entités de notre architecture et justification de la 3ème Forme Normale (3NF) 7

8 Triggers et Procédures Stockées **9**

8.1 Procédures Stockées 9

8.1.1 CalculateVulnerabilityScore 9

8.1.2 GenerateMonumentReport 9

8.2 Triggers 10

8.3 Diagramme de Flux des Triggers 10

8.4 Exemple de Code SQL des Triggers 11

9 Utilisation de l'Intelligence Artificielle dans le Projet **12**

10 Conclusion **14**

1 Introduction

Ce projet vise à développer un système de surveillance et de valorisation du patrimoine bâti de la ville de Taroudant, au Maroc. L'application permet aux autorités municipales de suivre l'état des monuments historiques et des remparts, d'identifier les risques structurels et de prendre des décisions éclairées pour la préservation du patrimoine culturel.

Le système offre une plateforme centralisée qui automatise la collecte des données d'inspection, fournit des analyses en temps réel et génère des rapports détaillés pour les décideurs.

2 Présentation du projet

Le projet Taroudant Heritage Shield est un système complet de gestion du patrimoine comprenant :

- Une application web moderne avec React et Vite
- Un backend API développé avec FastAPI (Python)
- Une base de données MySQL avec procédures stockées et triggers
- Des fonctionnalités de sécurité avancées (JWT, RBAC, AES-256)
- Un système de scoring automatique de la vulnérabilité structurelle
- Des alertes pour les autorités en cas de risque critique

Le système permet de :

- Surveiller les monuments historiques et les remparts de Taroudant
- Scorer automatiquement la vulnérabilité structurelle à partir des inspections sur le terrain
- Alerter les autorités municipales lorsqu'un risque critique est détecté
- Fournir des rapports d'experts chiffrés pour les décideurs
- Afficher un catalogue public de l'état de santé des monuments pour les citoyens

3 Architecture du système

L'architecture du système suit une approche moderne en trois couches :

3.1 Frontend

- **Framework** : React 18 avec TypeScript
- **Build tool** : Vite pour un développement rapide
- **Styling** : Tailwind CSS avec composants Radix UI
- **State Management** : React Query pour la gestion d'état serveur
- **Formulaires** : React Hook Form avec validation Zod
- **Visualisation** : Recharts pour les graphiques, Leaflet pour les cartes
- **3D** : React Three Fiber pour les visualisations 3D

3.2 Backend

- **Framework** : FastAPI (Python) avec Uvicorn
- **Authentification** : JWT avec tokens de rafraîchissement
- **Sql Injection** : Préparer les requetes pour que l'exécution soit etre comme une donnée non un code exécutable.
- **Sécurité** : bcrypt pour les mots de passe, AES-256 pour les données sensibles
- **Email** : fastapi-mail pour les notifications
- **PDF** : fpdf2 pour la génération de rapports

3.3 Base de données

- **SGBD** : MySQL 8.0
- **Modélisation** : Schéma en 3NF (Troisième Forme Normale)
- **Procédures stockées** : Pour les opérations complexes
- **Triggers** : Pour la gestion automatique des scores de vulnérabilité
- **RBAC** : Gestion des rôles et permissions

4 Modèle Conceptuel de Données (DCM)

Le schéma de la base de données comprend les entités principales suivantes :

- **roles** : Définit les rôles pour le contrôle d'accès basé sur les rôles (RBAC)
- **users** : Stocke les utilisateurs système (administrateurs, inspecteurs, autorités)
- **monument_categories** : Normalise la classification des monuments (murs, portes, mosquées, etc.)
- **monuments** : Entité centrale représentant chaque monument patrimonial surveillé
- **inspections** : Enregistre les inspections effectuées sur les monuments
- **cracks** : Suit les fissures identifiées lors des inspections
- **crack_photos** : Stocke les photos des fissures
- **reports** : Rapports générés pour les décideurs
- **notifications** : Système d'alertes pour les utilisateurs
- **access_requests** : Gestion des demandes d'accès au système
- **inspector_assignments** : Attribution des inspecteurs aux monuments

5 Points de terminaison de l'API

L'API RESTful expose les endpoints suivants :

5.1 Authentification

- `POST /api/login` – Connexion utilisateur
- `POST /api/logout` – Déconnexion
- `POST /api/token/refresh` – Rafraîchissement du token
- `GET /api/verify-token` – Vérification du token

— `POST /api/complete-account` – Complétion du compte
— `GET /api/me` – Profil utilisateur connecté

5.2 Monuments

— `GET /api/monuments` – Liste des monuments
— `GET /api/monuments/:id` – Détails d'un monument
— `POST /api/monuments` – Créer un monument
— `PUT /api/monuments/:id` – Modifier un monument
— `DELETE /api/monuments/:id` – Supprimer un monument
— `GET /api/monuments/:id/inspections` – Inspections d'un monument
— `GET /api/monuments/:id/inspections/public` – Inspections publiques
— `GET /api/monuments/:id/image` – Image d'un monument

5.3 Inspections

— `GET /api/inspections` – Liste des inspections
— `POST /api/inspections` – Créer une inspection
— `GET /api/inspections/:id` – Détails d'une inspection
— `GET /api/inspections/:id/detail` – Détails complets

5.4 Fissures (Cracks)

— `POST /api/cracks` – Créer une fissure
— `POST /api/cracks/:id/photo` – Ajouter une photo
— `GET /api/cracks/:id/photos` – Photos d'une fissure
— `GET /api/cracks/:id/image` – Image d'une fissure
— `GET /api/cracks/inspection/:id` – Fissures par inspection

5.5 Rapports

— `POST /api/reports` – Générer un rapport
— `GET /api/reports` – Liste des rapports
— `GET /api/reports/:id` – Détails d'un rapport
— `GET /api/reports/:id/pdf` – Télécharger PDF

5.6 Utilisateurs et Administration

— `POST /api/users` – Créer un utilisateur
— `GET /api/users` – Liste des utilisateurs
— `GET /api/admin/audit-logs` – Journaux d'audit
— `GET /api/admin/stats` – Statistiques

5.7 Notifications et Affectations

— `GET /api/notifications` – Notifications utilisateur
— `GET /api/notifications/triggered` – Alertes déclenchées

— `POST /api/assignments` – Affecter un inspecteur
— `GET /api/assignments` – Liste des affectations
— `PUT /api/assignments/:id/status` – Mettre à jour le statut

5.8 Demandes d'accès

— `POST /api/access-requests` – Demander l'accès
— `GET /api/access-requests` – Liste des demandes
— `POST /api/access-requests/:id/approve` – Approuver
— `POST /api/access-requests/:id/reject` – Rejeter

5.9 Analytiques

— `GET /api/analytics` – Données analytiques

6 Sécurité

Le système implémente plusieurs couches de sécurité :

— **Authentification JWT** : Tokens d'accès avec expiration, tokens de rafraîchissement
— **RBAC** : Contrôle d'accès basé sur les rôles (admin, inspector, authority, viewer)
— **Encryption** : AES-256 pour les données sensibles, bcrypt pour les mots de passe
— **CORS** : Configuré pour permettre les credentials (cookies httpOnly)
— **Validation** : Pydantic pour la validation des entrées
— **Audit** : Journaux d'audit pour les actions sensibles

7 Diagrammes Techniques

7.1 Entités de notre architecture et justification de la 3ème Forme Normale (3NF)

La base de données a été conçue selon la **Troisième Forme Normale (3NF)** pour éliminer la redondance et garantir l'intégrité des données. Voici les justifications :

Table 1 – Analyse de normalisation par table

Table	Dépendances	Justification 3NF
`roles`	role_id → role_name, description	Clé primaire unique. Aucun attribut ne dépend d'un autre attribut non-clé.
`users`	id_user → tous les attributs	role_id est une clé étrangère. Tous les attributs dépendent uniquement de la clé primaire.
`monument_categories`	category_id → category_name, description	Table de référence sans dépendance transitive. Normalisation respectée.
`monuments`	monument_id → tous les attributs	category_id est une clé étrangère. Aucun attribut non-clé ne dépend d'un autre attribut non-clé.
`inspections`	inspection_id → tous les attributs	monument_id et inspector_id sont des clés étrangères. Pas de dépendance transitive.
`cracks`	crack_id → tous les attributs	inspection_id est une clé étrangère. Les attributs décrivent uniquement une fissure.
`vulnerability_scores`	score_id → tous les attributs	monument_id et inspection_id sont des clés étrangères. Score dépend uniquement de la clé primaire.
`reports`	report_id → tous les attributs	Clés étrangères vers plusieurs tables. Pas de dépendance transitive. 3NF respectée.
`notifications`	notification_id → tous les attributs	Les clés étrangères assurent les relations. Aucun attribut non-clé dépend d'un autre.

Table	Dépendances	Justification 3NF
`inspector_assignments`	assignment_id → tous les attributs	Toutes les colonnes dépendent uniquement de la clé primaire. Table en 3NF.
`audit_logs`	log_id → tous les attributs	user_id est une clé étrangère. Les données décrivent une action unique.
`access_requests`	id → tous les attributs	requested_role_id et reviewed_by_id sont des clés étrangères. Pas de dépendance transitive.

Principales décisions de normalisation :

- **Extraction des catégories :** Les catégories de monuments sont isolées dans `monument_categorie` pour éviter la répétition des noms de catégories.
- **Séparation des rôles :** Les rôles utilisateurs (admin, inspector, authority) sont normalisés dans `roles` pour faciliter la gestion RBAC.
- **Élimination des groupes répétitifs :** Les fissures (`cracks`) sont stockées dans une table séparée liée aux inspections, plutôt que comme attributs multiples.
- **Scores de vulnérabilité :** Les scores calculés sont stockés dans leur propre table avec référence aux inspections, permettant l'historisation.

8 Triggers et Procédures Stockées

Cette section décrit les **triggers** et **procédures stockées** implémentés dans la base de données MySQL, avec leurs objectifs respectifs.

8.1 Procédures Stockées

8.1.1 CalculateVulnerabilityScore

Table 2 – Spécification de la procédure CalculateVulnerabilityScore

Attribut	**Valeur**
Nom	`CalculateVulnerabilityScore`
Paramètre	`p_inspection_id INT`
Objectif	Calculer automatiquement le score de vulnérabilité d'un monument basé sur son âge, les fissures détectées et la catégorie du monument.
Formule	$(\text{crack_score} + \text{age_score}) \times \text{category_factor}$

Détails du calcul :

- **Crack Score :** Somme pondérée des fissures (mineure=1, modérée=3, majeure=7, critique=15), multipliée par un **facteur de longueur normalisé** (longueur divisée par 10, 20 ou 30 cm selon la sévérité pour obtenir un ratio d'impact).
- **Age Multiplier :** $1.0 + \min(\text{age}/1000, 0.5)$ — les fissures sur les monuments anciens sont plus critiques.
- **Category Factor :** Ajustement selon le type (Rempart=0.8, Kasbah=0.85, Porte=0.9, Mosquée=1.0, Maison=1.2).
- **Age Score :** $\min(\lfloor \text{age}/20 \rfloor, 10)$ — chaque génération (20 ans) ajoute 1 point de risque latent, capé à 10 points (10% du risque total) pour maintenir la priorité sur les dégâts réels.

8.1.2 GenerateMonumentReport

Table 3 – Spécification de la procédure GenerateMonumentReport

Attribut	**Valeur**
Nom	`GenerateMonumentReport`
Paramètres	`p_monument_id INT, p_inspection_id INT, p_generated_by INT`
Objectif	Générer un rapport complet encrypté avec les informations du monument, de l'inspection et des statistiques de fissures.
Encryption	AES-256 avec clé dérivée SHA2

Sections du rapport généré :

1. Informations sur le monument (nom, localisation, année de construction)
2. Synthèse de l'inspection (date, état global, observations)
3. Fissures observées (total, répartition par sévérité)
4. Vulnérabilité (scores âge/fissures/total, niveau de risque)
5. Recommandation selon le niveau de risque

8.2 Triggers

Table 4 – Liste des triggers et leurs objectifs

Nom du trigger	Événement	Objectif
`after_crack_insert`	AFTER INSERT sur `cracks`	Déclenche automatiquement le recalcul du score de vulnérabilité lorsqu'une nouvelle fissure est ajoutée, maintient à jour l'évaluation du risque en temps réel.
`after_score_insert`	AFTER INSERT sur `vulnerability_scores`	Génère des notifications automatiques pour les autorités (rôle`authority`) lorsqu'un score de risque `high` ou `critical` est inséré.
`after_report_insert`	AFTER INSERT sur `reports`	Crée une entrée dans `audit_logs` pour tracer la génération de rapports : qui, quand et pour quel monument.
`after_access_request_reviewed`	AFTER UPDATE sur `access_requests`	Journalise dans `audit_logs` les décisions d'approbation/-rejet des demandes d'accès avec la note de révision.

8.3 Diagramme de Flux des Triggers

La figure 1 illustre le flux d'exécution des triggers dans le système.

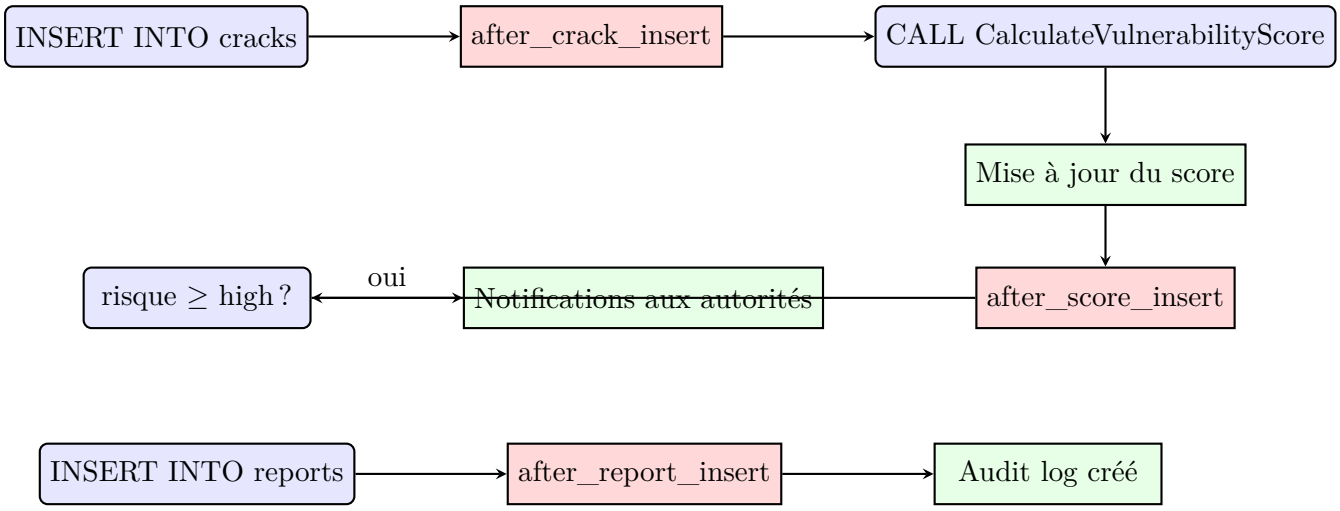

Figure 1 – Flux d'exécution des triggers dans le système

8.4 Exemple de Code SQL des Triggers

Listing 1 – Trigger after_crack_insert - Recalcul automatique du score

```
CREATE TRIGGER after_crack_insert
AFTER INSERT ON cracks
FOR EACH ROW
BEGIN
  -- Delegate scoring logic to the stored procedure
  CALL CalculateVulnerabilityScore(NEW.inspection_id);
END;
```

Listing 2 – Trigger after_score_insert - Notifications automatiques

```
CREATE TRIGGER after_score_insert
AFTER INSERT ON vulnerability_scores
FOR EACH ROW
BEGIN
  DECLARE v_authority_role_id INT;
  DECLARE v_monument_name VARCHAR(150);

  -- Only generate alerts for high or critical risks
  IF NEW.risk_level IN ('high', 'critical') THEN
    -- Find authority role and monument name
    SELECT role_id INTO v_authority_role_id
    FROM roles WHERE role_name = 'authority' LIMIT 1;

    SELECT name INTO v_monument_name
    FROM monuments WHERE monument_id = NEW.monument_id;

    -- Insert notifications for all authority users
    INSERT INTO notifications (monument_id,
       triggered_by_inspection,
      recipient_id, message, severity, is_read, sent_at)
    SELECT NEW.monument_id, NEW.inspection_id, u.id_user,
      CONCAT('ALERTE: Monument [', v_monument_name,
             '] - Niveau de risque: ', NEW.risk_level),
      NEW.risk_level, FALSE, NOW()
    FROM users u WHERE u.role_id = v_authority_role_id;
  END IF;
END;
```

9 Utilisation de l'Intelligence Artificielle dans le Projet

Afin de réaliser ce projet de manière efficiente, nous avons fait appel à un ensemble d'outils et de modèles d'Intelligence Artificielle. Pour garantir la qualité de la base de code, nous avons exploité la fonctionnalité **« Skills »** d'Antigravity permettant à l'agent de s'appuyer sur des instructions métiers spécifiques. De plus, nous avons adopté rigoureusement notre méthode **RCTF** (Role, Context, Task, Format) lors de la formulation de nos requêtes afin de garder l'agent concentré dans le même contexte de développement, ce qui a drastiquement amélioré ses performances.

Outils d'Édition et de Test

Cursor & Antigravity

Antigravity a été au cœur de notre développement agentique, tandis que Cursor simplifiait l'interaction dans l'éditeur de code. Un atout majeur d'Antigravity fut son *sub-agent browser*. Nous l'avons largement sollicité pour automatiser et tester dynamiquement l'interface utilisateur web ainsi que ses comportements en simulation de production.

Recherche et Données

Perplexity AI

Nous avons utilisé Perplexity pour rechercher les informations historiques et réaliser une géolocalisation détaillée des monuments.

Avantage : Fournit des sources fiables et permet une vérification rapide de vraies données historiques.

Inconvénient : Ne génère ni n'insère de code directement au sein de notre architecture logicielle.

Modèles d'IA exploités dans Antigravity

Pour couvrir nos différents besoins technologiques (Backend, UI, tests, requêtes SQL), nous avons utilisé divers modèles de pointe à travers Antigravity :

Gemini 3.1 Pro (High)

Avantage : Excellente capacité de raisonnement sur l'architecture globale, offrant des implémentations de très haute qualité.
Inconvénient : Alors les limites observées restent minimes, comme une légère latence lorsque le contexte fourni est extrêmement long.

Google Flash

Avantage : Contient une bonne vitesse de réponse, préférable de l'utiliser pour les simples.
Inconvénient : Parmis ses limites est manque de profondeur mais c'est logique à cause de l'utiliser pour les taches complexes.

Claude Sonnet 4.6 (thinking)

Avantage : Le processus de réflexion **thinking** préalable aide grandement le modèle à résoudre et déboguer les algorithmes ardus avant la rédaction du code ainsi il est ainsi utilisé pour founir des prompts bien structurés.
Inconvénient : Ses contraintes sont limitées, principalement liées à la gestion de vastes fenêtres de longs historiques ou logs. À noter que ses suggestions de code peuvent enrichir et affiner les résultats produits par Gemini Pro.

10 Conclusion

Le projet Taroudant Heritage Shield fournit une solution complète et moderne pour la gestion du patrimoine de Taroudant. En combinant une architecture robuste (React/-FastAPI/MySQL) avec des fonctionnalités avancées de sécurité et d'analyse, le système permet aux autorités municipales de prendre des décisions éclairées pour la préservation du patrimoine culturel.

Les principales contributions du projet incluent :

— L'automatisation du processus d'inspection
— Le scoring automatique de la vulnérabilité
— La génération de rapports chiffrés
— Un système d'alertes en temps réel
— Une interface publique pour les citoyens

Références

[1] FastAPI Documentation, https://fastapi.tiangolo.com/

[2] React Documentation, https://react.dev/

[3] MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/

[4] Tailwind CSS Documentation, https://tailwindcss.com/docs

[5] Vite Documentation, https://vitejs.dev/guide/